

Task 4 is mostly writing tests for the cases the assignment already gives you: valid traffic, modified ciphertext, modified header, replay, reflection, and bad handshake authentication. Then you write the 500–700 word security note.  HW2-2

If you stay focused, you can probably finish both tonight.

[walk me through task 3](#)

Task 3 is much more straightforward than Task 2. You're building a **record layer** that takes plaintext messages and protects them using the keys you derived in Task 2.

The assignment wants a `secure_record.py` file with two main functions:

`</>` Python

```
seal()
open_record()
```

The protected record format is:

`</>` Plain text

```
header
iv
ciphertext
tag
```

where:

`</>` Plain text

```
header =
version(1 byte)
|| direction(1 byte)
|| sequence(8 bytes)
|| message_type(1 byte)
|| ciphertext_length(4 bytes)

iv = session_id(8 bytes) || sequence(8 bytes)

ciphertext = AES-256-CTR(K_enc, iv, plaintext)
```

```
tag = HMAC-SHA-256(K_mac, header || iv || ciphertext)
```

That is the exact format required by the homework.  HW2-2

Step 1: Create `secure_record.py`

From your `hw2` directory:

</> Bash

```
nano secure_record.py
```

Start with:

</> Python

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import hashes, hmac
from cryptography.exceptions import InvalidSignature
```

You need AES-CTR and HMAC-SHA-256.

Step 2: Understand the header

Your header has five pieces:

</> Plain text

version	1 byte
direction	1 byte
sequence	8 bytes
message_type	1 byte
ciphertext length	4 bytes

You could write a helper:

</> Python

```
def build_header(version, direction, sequence, message_type, ciphertext_length):

    version_bytes = version.to_bytes(1, "big")
```

```

direction_bytes = direction.to_bytes(1, "big")
sequence_bytes = sequence.to_bytes(8, "big")
message_type_bytes = message_type.to_bytes(1, "big")
length_bytes = ciphertext_length.to_bytes(4, "big")

return (
    version_bytes
    + direction_bytes
    + sequence_bytes
    + message_type_bytes
    + length_bytes
)

```

For example:

</> Python

```

version = 1
direction = 0
sequence = 0
message_type = 1

```

could represent:

</> Plain text

```

version 1
gateway → node
message #0
normal command

```

The assignment doesn't prescribe your numeric values for direction/message type, so you can define constants such as:

</> Python

```

GATEWAY_TO_NODE = 0
NODE_TO_GATEWAY = 1

```

Step 3: Build the IV

The assignment requires:

</> Plain text

```
iv = session_id || sequence
```

The session ID is 8 bytes and sequence is 8 bytes, giving a 16-byte AES-CTR IV.



HW2-2

So make:

</> Python

```
def make_iv(session_id, sequence):  
    sequence_bytes = sequence.to_bytes(8, "big")  
    return session_id + sequence_bytes
```

If:

</> Python

```
len(session_id) == 8
```

then:

</> Python

```
len(iv) == 16
```

Step 4: Write the AES-CTR helper

This should look familiar from Task 1:

</> Python

```
def encrypt_ctr(key, iv, plaintext):  
  
    cipher = Cipher(  
        algorithms.AES(key),  
        modes.CTR(iv)  
    )  
  
    encryptor = cipher.encryptor()
```

```
return encryptor.update(plaintext) + encryptor.finalize()
```

And eventually you'll also need decryption:

</> Python

```
def decrypt_ctr(key, iv, ciphertext):  
  
    cipher = Cipher(  
        algorithms.AES(key),  
        modes.CTR(iv)  
    )  
  
    decryptor = cipher.decryptor()  
  
    return decryptor.update(ciphertext) + decryptor.finalize()
```

Step 5: Make the HMAC

Task 1 had no integrity protection.

Task 3 fixes that by computing:

</> Plain text

```
HMAC(K_mac, header || iv || ciphertext)
```

Make a helper:

</> Python

```
def create_tag(mac_key, data):  
  
    h = hmac.HMAC(mac_key, hashes.SHA256())  
    h.update(data)  
  
    return h.finalize()
```

The output will be 32 bytes because SHA-256 produces 256 bits.

Step 6: Build `seal()`

`seal()` is the sender.

Its job is:

`</>` Plain text

```
plaintext
↓
build IV
↓
encrypt
↓
build header
↓
calculate MAC
↓
return complete record
```

Something like:

`</>` Python

```
def seal(
    enc_key,
    mac_key,
    session_id,
    sequence,
    direction,
    message_type,
    plaintext
):
```

Inside, first create the IV:

`</>` Python

```
iv = make_iv(session_id, sequence)
```

Then encrypt:

`</>` Python

```
ciphertext = encrypt_ctr(  
    enc_key,  
    iv,  
    plaintext  
)
```

Then build the header:

</> Python

```
header = build_header(  
    1,  
    direction,  
    sequence,  
    message_type,  
    len(ciphertext)  
)
```

Then authenticate everything:

</> Python

```
tag = create_tag(  
    mac_key,  
    header + iv + ciphertext  
)
```

Then combine it:

</> Python

```
record = header + iv + ciphertext + tag
```

and:

</> Python

```
return record
```

Conceptually:

</> Plain text

```
seal("hello")
```

↓

```
header
```

```
+
```

```
IV
```

```
+
```

```
encrypted "hello"
```

```
+
```

```
HMAC
```

Step 7: Sequence numbers

This part is important.

The assignment requires:

sequence numbers begin at zero in each direction.



HW2-2

So gateway → node might have:

</> Plain text

```
0
```

```
1
```

```
2
```

```
3
```

while node → gateway independently has:

</> Plain text

```
0
```

```
1
```

```
2
```

```
3
```

Don't use one shared sequence number.

Later you might have:

</> Python

```
gateway_send_sequence = 0
node_send_sequence = 0
```

After successfully sending:

</> Python

```
gateway_send_sequence += 1
```

The reason is replay protection.

Step 8: `open_record()` is the receiver

This is where you have to be careful.

The assignment explicitly requires:

verify HMAC before decrypting.  HW2-2

So your order should be:

</> Plain text

```
receive bytes
↓
parse header
↓
check direction
↓
check expected sequence
↓
verify HMAC
↓
ONLY THEN decrypt
```

Do **not** do:

</> Plain text

```
decrypt
↓
check HMAC
```

Step 9: Parse the record

Your header length is:

</> Plain text

$1 + 1 + 8 + 1 + 4 = 15$ bytes

Then the IV is:

</> Plain text

16 bytes

and the tag is:

</> Plain text

32 bytes

So you can extract:

</> Python

```
header = record[:15]
```

Then fields:

</> Python

```
version = header[0]
direction = header[1]
```

```
sequence = int.from_bytes(
    header[2:10],
    "big"
```

```
)

message_type = header[10]

ciphertext_length = int.from_bytes(
    header[11:15],
    "big"
)
```

Then:

</> Python

```
iv = record[15:31]
```

Ciphertext begins at byte 31:

</> Python

```
ciphertext_start = 31
ciphertext_end = ciphertext_start + ciphertext_length

ciphertext = record[ciphertext_start:ciphertext_end]
```

And the remaining 32 bytes:

</> Python

```
tag = record[ciphertext_end:]
```

Step 10: Check sequence number

Suppose the receiver expects:

</> Python

```
expected_sequence = 3
```

Then:

</> Python

```
if sequence != expected_sequence:
    raise ValueError("Unexpected sequence number")
```

That prevents replay.

For example:

</> Plain text

```
receiver expects #3
attacker sends old record #1
```

```
1 != 3
→ reject
```

This directly satisfies the requirement that the receiver only accept the exact next sequence number.

 HW2-2

Step 11: Check direction

Suppose a gateway-to-node record uses:

</> Python

```
direction = 0
```

and the node expects 0 .

Then:

</> Python

```
if direction != expected_direction:
    raise ValueError("Wrong direction")
```

This is how you'll later catch the reflection test.

Step 12: Verify the HMAC

This is crucial.

You should authenticate:

</> Plain text

```
header || iv || ciphertext
```

using the MAC key.

Instead of manually comparing tags with `==`, use the library's verification:

</> Python

```
def verify_tag(mac_key, data, tag):  
  
    h = hmac.HMAC(mac_key, hashes.SHA256())  
    h.update(data)  
  
    h.verify(tag)
```

The library performs constant-time verification, which the assignment specifically requires.



HW2-2

Then inside `open_record()` :

</> Python

```
verify_tag(  
    mac_key,  
    header + iv + ciphertext,  
    tag  
)
```

If someone changed:

</> Plain text

```
header  
IV  
ciphertext
```

the HMAC won't match.

Step 13: Only decrypt after HMAC succeeds

Now:

</> Python

```
plaintext = decrypt_ctr(  
    enc_key,  
    iv,  
    ciphertext  
)
```

Then:

</> Python

```
return plaintext
```

This is the safe flow:

</> Plain text

```
tampered ciphertext  
    ↓  
HMAC verification fails  
    ↓  
NO DECRYPTION  
    ↓  
NO PLAINTEXT RETURNED
```

The assignment explicitly says no plaintext should be released after an error.  HW2-2

Your final structure

Your file will probably end up looking approximately like:

</> Python

```
# imports
```

```

GATEWAY_TO_NODE = 0
NODE_TO_GATEWAY = 1

def build_header(...):
    ...

def make_iv(...):
    ...

def encrypt_ctr(...):
    ...

def decrypt_ctr(...):
    ...

```

Then your successful flow becomes:

</> Plain text

Task 2
 derive:
 K_g2n_enc
 K_g2n_mac
 session_id

↓

Task 3

```

seal(
    K_g2n_enc,
    K_g2n_mac,
    session_id,
    sequence=0,
    ...
)

```

↓

protected record

The four big ideas you should understand for Task 3 are:

Confidentiality: AES-CTR hides the plaintext.